

```
In [1]: import sys  
sys.version
```

```
Out[1]: '3.11.7 | packaged by Anaconda, Inc. | (main, Dec 15 2023, 18:05:4  
7) [MSC v.1916 64 bit (AMD64)]'
```

work with numbers

```
In [4]: 3
```

```
Out[4]: 3
```

```
In [5]: 2
```

```
Out[5]: 2
```

```
In [2]: 3 + 2
```

```
Out[2]: 5
```

```
In [6]: 3 - 2
```

```
Out[6]: 1
```

```
In [7]: 3 * 2
```

```
Out[7]: 6
```

```
In [8]: 3 ** 2
```

```
Out[8]: 9
```

```
In [9]: 10 / 5
```

```
Out[9]: 2.0
```

```
In [10]: 10 // 5
```

```
Out[10]: 2
```

```
In [11]: # work with string
```

```
In [12]: nareshit
```

```
-----  
-----  
NameError                                Traceback (most recent call  
last)  
Cell In[12], line 1  
----> 1 nareshit  
  
NameError: name 'nareshit' is not defined
```

```
In [13]: 'nareshit'
```

```
Out[13]: 'nareshit'
```

```
In [14]: " nareshit "
```

```
Out[14]: ' nareshit '
```

```
In [15]: ''' naresh it '''
```

```
Out[15]: ' naresh it '
```

variable = object

```
In [16]: v = 5 #v - variable & 5 for value  
v
```

```
Out[16]: 5
```

```
In [17]: type(v)
```

```
Out[17]: int
```

```
In [19]: v1 = 'nit'  
v2
```

```
-----  
-----  
NameError                                Traceback (most recent call  
last)  
Cell In[19], line 2  
      1 v1 = 'nit'  
----> 2 v2  
  
NameError: name 'v2' is not defined
```

```
In [20]: v1
```

```
Out[20]: 'nit'
```

26 Th

```
In [2]: a = 5.5  
        type(a)
```

```
Out[2]: float
```

```
In [ ]: import sys  
        syst.version
```

```
In [3]: nit = 15  
        NIT
```

```
-----  
-----  
NameError                                Traceback (most recent call  
last)  
Cell In[3], line 2  
      1 nit = 15  
----> 2 NIT  
  
NameError: name 'NIT' is not defined
```

```
In [4]: nit
```

```
Out[4]: 15
```

```
In [5]: 1a = 67  
        1a
```

```
Cell In[5], line 1  
    1a = 67  
    ^  
SyntaxError: invalid decimal literal
```

```
In [6]: a1 = 67  
        a1
```

```
Out[6]: 67
```

```
In [7]: nit$ = 89  
        nit$
```

```
Cell In[7], line 1  
    nit$ = 89  
    ^  
SyntaxError: invalid syntax
```

```
In [9]: x_train, x_test, y_train, y_test = 80, 20, 70, 30
```

```
In [13]: x_train  
         x_test  
         y_train  
         y_test
```

```
Out[13]: 30
```

```
In [14]: print(x_train)
         print(x_test)
         print(y_train)
         print(y_test)
```

```
80
20
70
30
```

```
In [17]: import keyword
         keyword.kwlist
```

```
Out[17]: ['False',
          'None',
          'True',
          'and',
          'as',
          'assert',
          'async',
          'await',
          'break',
          'class',
          'continue',
          'def',
          'del',
          'elif',
          'else',
          'except',
          'finally',
          'for',
          'from',
          'global',
          'if',
          'import',
          'in',
          'is',
          'lambda',
          'nonlocal',
          'not',
          'or',
          'pass',
          'raise',
          'return',
          'try',
          'while',
          'with',
          'yield']
```

```
In [18]: if = 90
         if
```

Cell In[18], line 1

```
if = 90
  ^
```

SyntaxError: invalid syntax

```
In [19]: a10 = 78  
        a9 = 89
```

```
In [20]: print(a10)  
        print(a9)
```

```
78  
89
```

```
In [21]: del a10
```

```
In [22]: a10
```

```
-----  
NameError                                Traceback (most recent call  
last)  
Cell In[22], line 1  
----> 1 a10  
  
NameError: name 'a10' is not defined
```

```
In [23]: for = 90
```

```
Cell In[23], line 1  
    for = 90  
      ^  
SyntaxError: invalid syntax
```

```
In [24]: For = 90  
        For
```

```
Out[24]: 90
```

```
In [26]: a = True  
        a
```

```
Out[26]: True
```

```
In [28]: b = 'true'  
        b
```

```
Out[28]: 'true'
```

```
In [30]: pi = 3.17  
        pi
```

```
Out[30]: 3.17
```

```
In [31]: pi = 3.20  
        pi
```

```
Out[31]: 3.2
```

```
In [34]: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa = 90
         aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

```
Out[34]: 90
```

```
In [35]: a# = 100
```

```
Out[35]: True
```

Variable are completed

27th -- DATA TYPES

INT FLOAT BOOLEAN COMPLEX STRING

```
In [2]: i = 25 #value without decimal
         i
```

```
Out[2]: 25
```

```
In [3]: type(i)
```

```
Out[3]: int
```

```
In [4]: print(type(i))
```

```
<class 'int'>
```

```
In [5]: petrol = 109.50 #value with decimal
         petrol
```

```
Out[5]: 109.5
```

```
In [6]: type(petrol)
```

```
Out[6]: float
```

```
In [7]: b = true
         b
```

```
-----
NameError                                Traceback (most recent call
l last)
Cell In[7], line 1
----> 1 b = true
      2 b

NameError: name 'true' is not defined
```

```
In [8]: b = True  
b
```

```
Out[8]: True
```

```
In [9]: b1 = False  
b1
```

```
Out[9]: False
```

```
In [10]: True + False
```

```
Out[10]: 1
```

```
In [11]: True - True
```

```
Out[11]: 0
```

```
In [12]: True * False
```

```
Out[12]: 0
```

```
In [13]: False / True
```

```
Out[13]: 0.0
```

```
In [14]: False // True
```

```
Out[14]: 0
```

```
In [15]: True/False
```

```
-----  
-----  
ZeroDivisionError                                Traceback (most recent call  
last)  
Cell In[15], line 1  
----> 1 True/False  
  
ZeroDivisionError: division by zero
```

```
In [16]: c1 = 10 + 20j  
c1
```

```
Out[16]: (10+20j)
```

```
In [17]: type(c1)
```

```
Out[17]: complex
```

```
In [18]: c1.real
```

```
Out[18]: 10.0
```

```
In [21]: c1.imaginary
```

```
-----  
-----  
AttributeError                                Traceback (most recent call  
last)  
Cell In[21], line 1  
----> 1 c1.imaginary  
  
AttributeError: 'complex' object has no attribute 'imaginary'
```

```
In [22]: c1.imag
```

```
Out[22]: 20.0
```

```
In [23]: c1
```

```
Out[23]: (10+20j)
```

```
In [24]: c2 = 20 + 30j
```

```
In [25]: print(c1)  
         print(c2)
```

```
(10+20j)  
(20+30j)
```

```
In [26]: c1 + c2
```

```
Out[26]: (30+50j)
```

```
In [27]: c1 - c2
```

```
Out[27]: (-10-10j)
```

```
In [28]: c2 - c1
```

```
Out[28]: (10+10j)
```

```
In [29]: print(c1)  
         print(c2)
```

```
(10+20j)  
(20+30j)
```

```
In [33]: c3 = 20+ 15i
```

```
Cell In[33], line 1  
      c3 = 20+ 15i  
              ^  
SyntaxError: invalid decimal literal
```

```
In [30]: c1 * c2
```

```
Out[30]: (-400+700j)
```



```
In [32]: s = 'nareshit'  
s
```

```
Out[32]: 'nareshit'
```

```
In [34]: s1 = "naresh it"  
s1
```

```
Out[34]: 'naresh it'
```

```
In [37]: s2 = '''naresh  
          it'''  
s2
```

```
Out[37]: 'naresh\n      it'
```

```
In [39]: s
```

```
Out[39]: 'nareshit'
```

string slicing[:]

```
In [40]: s
```

```
Out[40]: 'nareshit'
```

```
In [41]: s[:]
```

```
Out[41]: 'nareshit'
```

```
In [43]: s[4] # forward indexin
```

```
Out[43]: 's'
```

```
In [44]: s
```

```
Out[44]: 'nareshit'
```

```
In [45]: s[-4] #backward indxing
```

```
Out[45]: 's'
```

```
In [46]: b
```

```
Out[46]: True
```

```
In [47]: int(True)
```

```
Out[47]: 1
```

```
In [48]: int(False)
```

```
Out[48]: 0
```

```
In [49]: True + False
```

```
Out[49]: 1
```

```
In [50]: True
```

```
Out[50]: True
```

```
In [51]: s
```

```
Out[51]: 'nareshit'
```

```
In [52]: s[1:7]
```

```
Out[52]: 'areshi'
```

```
In [53]: s
```

```
Out[53]: 'nareshit'
```

```
In [54]: s[10]
```

```
-----  
-----  
IndexError                                Traceback (most recent call  
last)  
Cell In[54], line 1  
----> 1 s[10]  
  
IndexError: string index out of range
```

```
In [56]: s
```

```
Out[56]: 'nareshit'
```

```
In [57]: len(s)
```

```
Out[57]: 8
```

python data types are completed

type casting

```
In [1]: int(2.3) #cast from float to int
```

Out[1]: 2

In [2]: `int(2.3, 3.0)`

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[2], line 1  
----> 1 int(2.3, 3.0)  
  
TypeError: 'float' object cannot be interpreted as an integer
```

In [3]: `int(True) #cast from bool to int`

Out[3]: 1

In [4]: `int(False)`

Out[4]: 0

In [5]: `True`

Out[5]: True

In [6]: `True + True`

Out[6]: 2

In [7]: `int(1+2j)`

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[7], line 1  
----> 1 int(1+2j)  
  
TypeError: int() argument must be a string, a bytes-like object or a  
real number, not 'complex'
```

In [8]: `int('10')`

Out[8]: 10

In [9]: `int('ten')`

```
-----  
-----  
ValueError                                Traceback (most recent call  
last)  
Cell In[9], line 1  
----> 1 int('ten')  
  
ValueError: invalid literal for int() with base 10: 'ten'
```

```
In [10]: float(10)
```

```
Out[10]: 10.0
```

```
In [11]: float(10, 20)
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[11], line 1  
----> 1 float(10, 20)  
  
TypeError: float expected at most 1 argument, got 2
```

```
In [12]: float(True)
```

```
Out[12]: 1.0
```

```
In [13]: float(False)
```

```
Out[13]: 0.0
```

```
In [14]: float(1+2j)
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[14], line 1  
----> 1 float(1+2j)  
  
TypeError: float() argument must be a string or a real number, not 'complex'
```

```
In [15]: float('10')
```

```
Out[15]: 10.0
```

```
In [16]: float('ten')
```

```
-----  
-----  
ValueError                                Traceback (most recent call  
last)  
Cell In[16], line 1  
----> 1 float('ten')  
  
ValueError: could not convert string to float: 'ten'
```

```
In [17]: True
```

```
Out[17]: True
```

```
In [18]: True + True
```

```
Out[18]: 2
```

```
In [19]: complex(10)
```

```
Out[19]: (10+0j)
```

```
In [20]: complex(10, 20)
```

```
Out[20]: (10+20j)
```

```
In [23]: complex(10,20,30,40,50)
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[23], line 1  
----> 1 complex(10,20,30,40,50)  
  
TypeError: complex() takes at most 2 arguments (5 given)
```

```
In [24]: complex(2.3)
```

```
Out[24]: (2.3+0j)
```

```
In [25]: complex(2.3, 4)
```

```
Out[25]: (2.3+4j)
```

```
In [26]: complex(True, True)
```

```
Out[26]: (1+1j)
```

```
In [27]: complex(False)
```

```
Out[27]: 0j
```

```
In [28]: complex('10')
```

Out[28]: (10+0j)

1st march

In [29]: `complex('10', '20')`

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[29], line 1  
----> 1 complex('10', '20')  
  
TypeError: complex() can't take second arg if first is a string
```

In [1]: `complex(10, '20')`

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[1], line 1  
----> 1 complex(10, '20')  
  
TypeError: complex() second arg can't be a string
```

In [2]: `bool(2)`

Out[2]: True

In [3]: `bool(0)`

Out[3]: False

In [4]: `bool(2, 5)`

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[4], line 1  
----> 1 bool(2, 5)  
  
TypeError: bool expected at most 1 argument, got 2
```

In [5]: `bool(3.2)`

Out[5]: True

In [6]: `bool(1 + 2j)`

Out[6]: True

```
In [7]: bool(0+0j)
```

```
Out[7]: False
```

```
In [8]: bool('hi')
```

```
Out[8]: True
```

```
In [9]: bool( )
```

```
Out[9]: False
```

```
In [10]: bool(*)
```

```
Cell In[10], line 1
    bool(*)
      ^
SyntaxError: invalid syntax
```

```
In [12]: str(7)
```

```
Out[12]: '7'
```

```
In [13]: str(3.4)
```

```
Out[13]: '3.4'
```

```
In [14]: str(1+2j)
```

```
Out[14]: '(1+2j)'
```

```
In [15]: str(TRUE)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[15], line 1
----> 1 str(TRUE)

NameError: name 'TRUE' is not defined
```

```
In [16]: str(True)
```

```
Out[16]: 'True'
```

Type casting we are completed

python operator

– arithmetic operator (+, -, *, /, //, **)
–

```
In [19]: x1, y1 = 10, 5
```

```
In [20]: x1 + y1
```

```
Out[20]: 15
```

```
In [21]: x1 - y1
```

```
Out[21]: 5
```

```
In [22]: x1 / y1
```

```
Out[22]: 2.0
```

```
In [23]: x1 // y1
```

```
Out[23]: 2
```

```
In [24]: x1 ** y1
```

```
Out[24]: 100000
```

assignment operator

```
In [27]: x = 2  
x
```

```
Out[27]: 2
```

```
In [28]: x = x + 2  
x
```

```
Out[28]: 4
```

```
In [29]: x += 2  
x
```

```
Out[29]: 6
```

```
In [30]: x += 4
```

```
In [31]: x
```

```
Out[31]: 10
```

```
In [32]: x -= 2  
x
```


Out[32]: 8

```
In [33]: x *= 3  
x
```

Out[33]: 24

```
In [34]: x /= 2  
x
```

Out[34]: 12.0

```
In [35]: x //= 2  
x
```

Out[35]: 6.0

unary operator

```
In [37]: n = 7 #negattion  
n
```

Out[37]: 7

```
In [38]: m = -(n)  
m
```

Out[38]: -7

```
In [40]: n
```

Out[40]: 7

```
In [41]: -n
```

Out[41]: -7

Realtional operator

```
In [43]: r1 = 5  
r2 = 6
```

```
In [44]: r1 > r2
```

Out[44]: False

```
In [46]: r1 < r2
```

Out[46]: True

```
In [47]: r1 == r2
```

```
Out[47]: False
```

```
In [48]: r1 != r2
```

```
Out[48]: True
```

```
In [49]: r1
```

```
Out[49]: 5
```

```
In [50]: r2
```

```
Out[50]: 6
```

```
In [51]: r3 = 6
```

```
In [52]: r1 == r3
```

```
Out[52]: False
```

```
In [53]: r2 == r3
```

```
Out[53]: True
```

```
In [54]: print(r1)
          print(r2)
          print(r3)
```

```
5
```

```
6
```

```
6
```

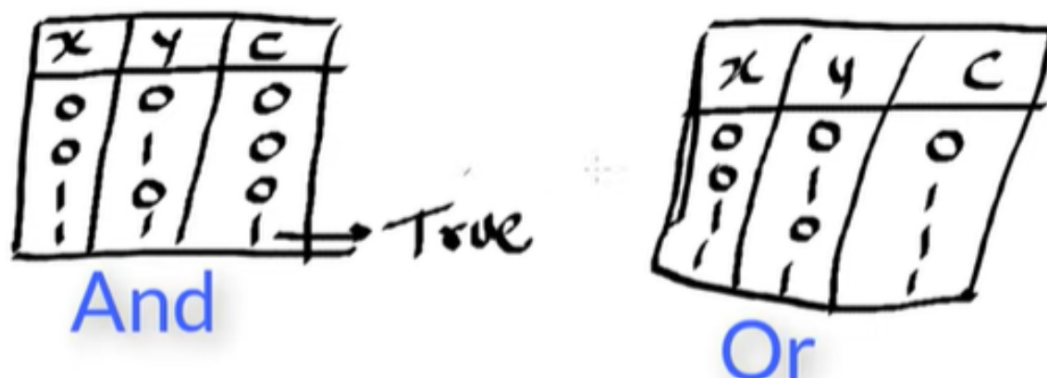
```
In [57]: r3 >= r2
```

```
Out[57]: True
```

```
In [58]: r2 <= r3
```

```
Out[58]: True
```

Truth Table



```
In [59]: a = 5
         b = 4
```

```
In [60]: a < 8 and b < 5
```

```
Out[60]: True
```

```
In [61]: a<8 or b<5
```

```
Out[61]: True
```

```
In [62]: print(a)
         print(b)
```

```
5
4
```

```
In [64]: b>5 or a<10
```

```
Out[64]: True
```

```
In [65]: x = False
         x
```

```
Out[65]: False
```

```
In [66]: not x
```

```
Out[66]: True
```

```
In [67]: y = True
         y
```

```
Out[67]: True
```

```
not y
```

python operator

3rd march

Datastruture - user will define the value more then one

- list
- tuple
- set
- **dict**

```
In [1]: l = []  
l
```

```
Out[1]: []
```

```
In [2]: len(l)
```

```
Out[2]: 0
```

```
In [3]: l.append(10)
```

```
In [4]: l
```

```
Out[4]: [10]
```

```
In [5]: len(l)
```

```
Out[5]: 1
```

```
In [6]: l
```

```
Out[6]: [10]
```

```
In [7]: l.append(20)  
l.append(30)  
l.append(40)  
l.append(40)
```

```
In [8]: l
```

```
Out[8]: [10, 20, 30, 40, 40]
```

```
In [9]: len(l)
```

```
Out[9]: 5
```

```
In [10]: l
```

```
Out[10]: [10, 20, 30, 40, 40]
```

```
In [11]: id(l)
```

```
Out[11]: 2243670320256
```

```
In [12]: print(type(l))
```

```
<class 'list'>
```

```
In [13]: a = True  
         type(a)
```

```
Out[13]: bool
```

```
In [14]: import keyword  
         keyword.kwlist
```

```
Out[14]: ['False',  
          'None',  
          'True',  
          'and',  
          'as',  
          'assert',  
          'async',  
          'await',  
          'break',  
          'class',  
          'continue',  
          'def',  
          'del',  
          'elif',  
          'else',  
          'except',  
          'finally',  
          'for',  
          'from',  
          'global',  
          'if',  
          'import',  
          'in',  
          'is',  
          'lambda',  
          'nonlocal',  
          'not',  
          'or',  
          'pass',  
          'raise',  
          'return',  
          'try',  
          'while',  
          'with',  
          'yield']
```

```
In [15]: len(keyword.kwlist)
```

```
Out[15]: 35
```

```
In [16]: l
```

```
Out[16]: [10, 20, 30, 40, 40]
```

```
In [17]: l[:]
```

```
Out[17]: [10, 20, 30, 40, 40]
```

```
In [18]: l[0]
```

```
Out[18]: 10
```

```
In [19]: l[1]
```

```
Out[19]: 20
```

```
In [20]: l[-3]
```

```
Out[20]: 30
```

```
In [21]: l
```

```
Out[21]: [10, 20, 30, 40, 40]
```

```
In [22]: l1 = l.copy()  
l1
```

```
Out[22]: [10, 20, 30, 40, 40]
```

```
In [23]: l == l1
```

```
Out[23]: True
```

```
In [24]: print(len(l))  
print(len(l1))
```

```
5  
5
```

```
In [25]: l1
```

```
Out[25]: [10, 20, 30, 40, 40]
```

```
In [26]: l1.append(2.3)  
l1.append(True)  
l1.append(1+2j)
```

```
In [27]: l1
```

```
Out[27]: [10, 20, 30, 40, 40, 2.3, True, (1+2j)]
```

```
In [28]: l1.append(50)
l1
```

```
Out[28]: [10, 20, 30, 40, 40, 2.3, True, (1+2j), 50]
```

```
In [29]: l
```

```
Out[29]: [10, 20, 30, 40, 40]
```

```
In [30]: l.count(10)
```

```
Out[30]: 1
```

```
In [31]: l.count(40)
```

```
Out[31]: 2
```

```
In [32]: l
```

```
Out[32]: [10, 20, 30, 40, 40]
```

```
In [33]: l.count(100)
```

```
Out[33]: 0
```

```
In [34]: l
```

```
Out[34]: [10, 20, 30, 40, 40]
```

```
In [35]: l1
```

```
Out[35]: [10, 20, 30, 40, 40, 2.3, True, (1+2j), 50]
```

```
In [36]: l2 = l1.copy()
```

```
In [37]: l2
```

```
Out[37]: [10, 20, 30, 40, 40, 2.3, True, (1+2j), 50]
```

```
In [38]: l2.remove(True)
```

```
In [39]: l2
```

```
Out[39]: [10, 20, 30, 40, 40, 2.3, (1+2j), 50]
```

```
In [40]: l2
```

```
Out[40]: [10, 20, 30, 40, 40, 2.3, (1+2j), 50]
```

```
In [41]: l2.remove(1+2j)
```

```
l2
```

```
Out[41]: [10, 20, 30, 40, 40, 2.3, 50]
```

```
In [42]: l2
```

```
Out[42]: [10, 20, 30, 40, 40, 2.3, 50]
```

```
In [43]: l2.clear()
```

```
In [44]: l2
```

```
Out[44]: []
```

```
In [45]: del l2
```

```
In [46]: l2
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[46], line 1
----> 1 l2

NameError: name 'l2' is not defined
```

4th march - list

```
In [48]: print(l)
         print(l1)
```

```
[10, 20, 30, 40, 40]
[10, 20, 30, 40, 40, 2.3, True, (1+2j), 50]
```

```
In [49]: print(len(l))
         print(len(l1))
```

```
5
9
```

```
In [50]: l
```

```
Out[50]: [10, 20, 30, 40, 40]
```

```
In [51]: for i in l:
         print(i)
```

```
10
20
30
40
40
```



```
In [52]: l.append([1,2,3,'hi']) #nested list
l
```

```
Out[52]: [10, 20, 30, 40, 40, [1, 2, 3, 'hi']]
```

```
In [53]: l
```

```
Out[53]: [10, 20, 30, 40, 40, [1, 2, 3, 'hi']]
```

```
In [54]: l.remove(40) #remove the element
```

```
In [55]: l
```

```
Out[55]: [10, 20, 30, 40, [1, 2, 3, 'hi']]
```

```
In [56]: l
```

```
Out[56]: [10, 20, 30, 40, [1, 2, 3, 'hi']]
```

```
In [57]: l[4]
```

```
Out[57]: [1, 2, 3, 'hi']
```

```
In [58]: l
```

```
Out[58]: [10, 20, 30, 40, [1, 2, 3, 'hi']]
```

```
In [59]: l.pop()
```

```
Out[59]: [1, 2, 3, 'hi']
```

```
In [60]: l
```

```
Out[60]: [10, 20, 30, 40]
```

```
In [62]: l1
```

```
Out[62]: [10, 20, 30, 40, 40, 2.3, True, (1+2j), 50]
```

```
In [63]: l1.pop()
```

```
Out[63]: 50
```

```
In [64]: l1
```

```
Out[64]: [10, 20, 30, 40, 40, 2.3, True, (1+2j)]
```

```
In [65]: l1.pop()
```

```
Out[65]: (1+2j)
```

```
In [67]: l1
```

```
Out[67]: [10, 20, 30, 40, 40, 2.3, True]
```

```
In [68]: l1.pop(-1)
```

```
Out[68]: True
```

```
In [69]: l1
```

```
Out[69]: [10, 20, 30, 40, 40, 2.3]
```

```
In [70]: l1.pop(3)
```

```
Out[70]: 40
```

```
In [71]: l1
```

```
Out[71]: [10, 20, 30, 40, 2.3]
```

```
In [72]: print(l)
         print(l1)
```

```
[10, 20, 30, 40]
```

```
[10, 20, 30, 40, 2.3]
```

```
In [73]: l
```

```
Out[73]: [10, 20, 30, 40]
```

```
In [75]: l.insert(35,3)
```

```
In [76]: l
```

```
Out[76]: [10, 20, 30, 40, 3]
```

```
In [77]: l.insert(3,35)
```

```
In [78]: l
```

```
Out[78]: [10, 20, 30, 35, 40, 3]
```

```
In [79]: l1
```

```
Out[79]: [10, 20, 30, 40, 2.3]
```

```
In [80]: l1.insert(15,1)
```

```
In [81]: l1
```

```
Out[81]: [10, 20, 30, 40, 2.3, 1]
```

```
In [82]: l1.insert(1, 15)
```

```
In [83]: l1
```

```
Out[83]: [10, 15, 20, 30, 40, 2.3, 1]
```

```
In [84]: l2 = []
```

```
In [85]: l2
```

```
Out[85]: []
```

```
In [87]: l2.extend(l1)
```

```
In [88]: l2
```

```
Out[88]: [10, 15, 20, 30, 40, 2.3, 1]
```

```
In [89]: l
```

```
Out[89]: [10, 20, 30, 35, 40, 3]
```

```
In [90]: l1
```

```
Out[90]: [10, 15, 20, 30, 40, 2.3, 1]
```

```
In [91]: l.extend(l1)
```

```
In [92]: l
```

```
Out[92]: [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [93]: print(l)
```

```
[10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [94]: print(l1)
```

```
[10, 15, 20, 30, 40, 2.3, 1]
```

```
In [95]: print(l2)
```

```
[10, 15, 20, 30, 40, 2.3, 1]
```

```
In [97]: l2.index(30)
```

```
Out[97]: 3
```

```
In [ ]: l2.
```

```
In [98]: l
```

```
Out[98]: [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [101... l.index(30)
```

Out[101... 2

In [102... l

Out[102... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]

In [103... l1

Out[103... [10, 15, 20, 30, 40, 2.3, 1]

In [104... l1.sort()

In [105... l1

Out[105... [1, 2.3, 10, 15, 20, 30, 40]

In [106... l1.sort(reverse=True) *#descending order*

In [107... l1

Out[107... [40, 30, 20, 15, 10, 2.3, 1]

In [108... l3 = [3, 100, 4]
l3

Out[108... [3, 100, 4]

In [109... l3.sort()

In [110... l3

Out[110... [3, 4, 100]

In [111... l3.sort(reverse = True)

In [112... l3

Out[112... [100, 4, 3]

In [113... l6 = [3, 5.6, 'a', 1+2j]

In [114... l6.sort()

```
-----  
-----  
TypeError                                Traceback (most recent call  
l last)  
Cell In[114], line 1  
----> 1 l6.sort()  
  
TypeError: '<' not supported between instances of 'str' and 'float'
```

```
In [115... l5 = ['z', 'm', 'n', 'b']  
l5
```

```
Out[115... ['z', 'm', 'n', 'b']
```

```
In [116... l5.sort()
```

```
In [117... l5
```

```
Out[117... ['b', 'm', 'n', 'z']
```

```
In [119... l1.reverse()
```

```
In [120... l1
```

```
Out[120... [1, 2.3, 10, 15, 20, 30, 40]
```

```
In [121... l1.reverse()
```

```
In [122... l1
```

```
Out[122... [40, 30, 20, 15, 10, 2.3, 1]
```

```
In [123... l
```

```
Out[123... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [124... l[::-1]
```

```
Out[124... [1, 2.3, 40, 30, 20, 15, 10, 3, 40, 35, 30, 20, 10]
```

5th

```
In [126... print(l)  
print(l1)  
print(l2)
```

```
[10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]  
[40, 30, 20, 15, 10, 2.3, 1]  
[10, 15, 20, 30, 40, 2.3, 1]
```

string list slicing (datatype)

```
In [128... s1 = 'nit'  
s1
```

```
Out[128... 'nit'
```

```
In [129... s1[0]
```

Out[129... 'n'

In [130... s1[1]

Out[130... 'i'

In [131... s1[2]

Out[131... 't'

In [132... s1[3]

```
-----  
-----  
IndexError                                Traceback (most recent call  
last)  
Cell In[132], line 1  
----> 1 s1[3]  
  
IndexError: string index out of range
```

In [133... s1

Out[133... 'nit'

In [134... **for** i **in** s1:
print(i)

n
i
t

In [135... s1

Out[135... 'nit'

list slicing

In [138... print(l)

[10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]

In [139... l[:]

Out[139... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]

In [140... l[0:8]

Out[140... [10, 20, 30, 35, 40, 3, 10, 15]

In [141... l

```
Out[141... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [142... l[3:]
```

```
Out[142... [35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [143... l
```

```
Out[143... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [144... l[:7]
```

```
Out[144... [10, 20, 30, 35, 40, 3, 10]
```

```
In [145... l
```

```
Out[145... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [146... l[0:20:5]
```

```
Out[146... [10, 3, 40]
```

```
In [147... l
```

```
Out[147... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [148... l[3:10:3]
```

```
Out[148... [35, 10, 30]
```

```
In [149... l
```

```
Out[149... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [150... l[::-1]
```

```
Out[150... [1, 2.3, 40, 30, 20, 15, 10, 3, 40, 35, 30, 20, 10]
```

```
In [151... l
```

```
Out[151... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [152... l[::-2]
```

```
Out[152... [1, 40, 20, 10, 40, 30, 10]
```

```
In [153... l
```

```
Out[153... [10, 20, 30, 35, 40, 3, 10, 15, 20, 30, 40, 2.3, 1]
```

```
In [154... l[::-3]
```

```
Out[154... [1, 30, 10, 35, 10]
```

```
In [155... l1
```

```
Out[155... [40, 30, 20, 15, 10, 2.3, 1]
```

```
In [156... l1[0]
```

```
Out[156... 40
```

```
In [157... l1[0] = 45
```

```
In [158... l1
```

```
Out[158... [45, 30, 20, 15, 10, 2.3, 1]
```

```
In [159... l1
```

```
Out[159... [45, 30, 20, 15, 10, 2.3, 1]
```

```
In [160... l1[-1] = 'nit'
```

```
In [161... l1
```

```
Out[161... [45, 30, 20, 15, 10, 2.3, 'nit']
```

```
In [162... l1[-1][0] # nested slicing
```

```
Out[162... 'n'
```

```
In [163... print(l1[-1][0])  
print(l1[-1][1])  
print(l1[-1][2])
```

```
n  
i  
t
```

```
In [164... l1
```

```
Out[164... [45, 30, 20, 15, 10, 2.3, 'nit']
```

```
In [165... l2
```

```
Out[165... [10, 15, 20, 30, 40, 2.3, 1]
```

```
In [168... len(l2)
```

```
Out[168... 7
```

```
In [166... l3
```



```
Out[166... [100, 4, 3]
```

```
In [169... l4 = l2 + l3
```

```
In [ ]: # list membership
```

```
In [170... l4
```

```
Out[170... [10, 15, 20, 30, 40, 2.3, 1, 100, 4, 3]
```

```
In [174... 15 in l4
```

```
Out[174... True
```

ENumerate

```
In [175... l1
```

```
Out[175... [45, 30, 20, 15, 10, 2.3, 'nit']
```

```
In [176... for i in l1:  
        print(i)
```

```
45  
30  
20  
15  
10  
2.3  
nit
```

```
In [177... for i in enumerate(l1):  
        print (i)
```

```
(0, 45)  
(1, 30)  
(2, 20)  
(3, 15)  
(4, 10)  
(5, 2.3)  
(6, 'nit')
```

```
In [178... l1
```

```
Out[178... [45, 30, 20, 15, 10, 2.3, 'nit']
```

```
In [179... l3
```

```
Out[179... [100, 4, 3]
```

```
In [180... all(l3)
```

Out[180... True

In [181... `any(l3)`

Out[181... True

`l3.append(0)` l3

In [183... `all(l3)`

Out[183... False

In [184... `any(l3)`

Out[184... True

List data strcuture is compleeted

In []:

In []:

In []:

In []: